Predicting Student Admissions with Neural Networks

In this notebook, we predict student admissions to graduate school at UCLA based on three pieces of data:

- GRE Scores (Test)
- GPA Scores (Grades)
- Class rank (1-4)

The dataset originally came from here: http://www.ats.ucla.edu/

Loading the data

To load the data and format it nicely, we will use two very useful packages called Pandas and Numpy. You can read on the documentation here:

- https://pandas.pydata.org/pandas-docs/stable/
- https://docs.scipy.org/

In [85]:

```
# Importing pandas and numpy
import pandas as pd
import numpy as np

# Reading the csv file into a pandas DataFrame
data = pd.read_csv('student_data.csv')

# Printing out the first 10 rows of our data
data.head()
```

Out[85]:

	admit	gre	gpa	rank
0	0	380	3.61	3
1	1	660	3.67	3
2	1	800	4.00	1
3	1	640	3.19	4
4	0	520	2.93	4

Plotting the data

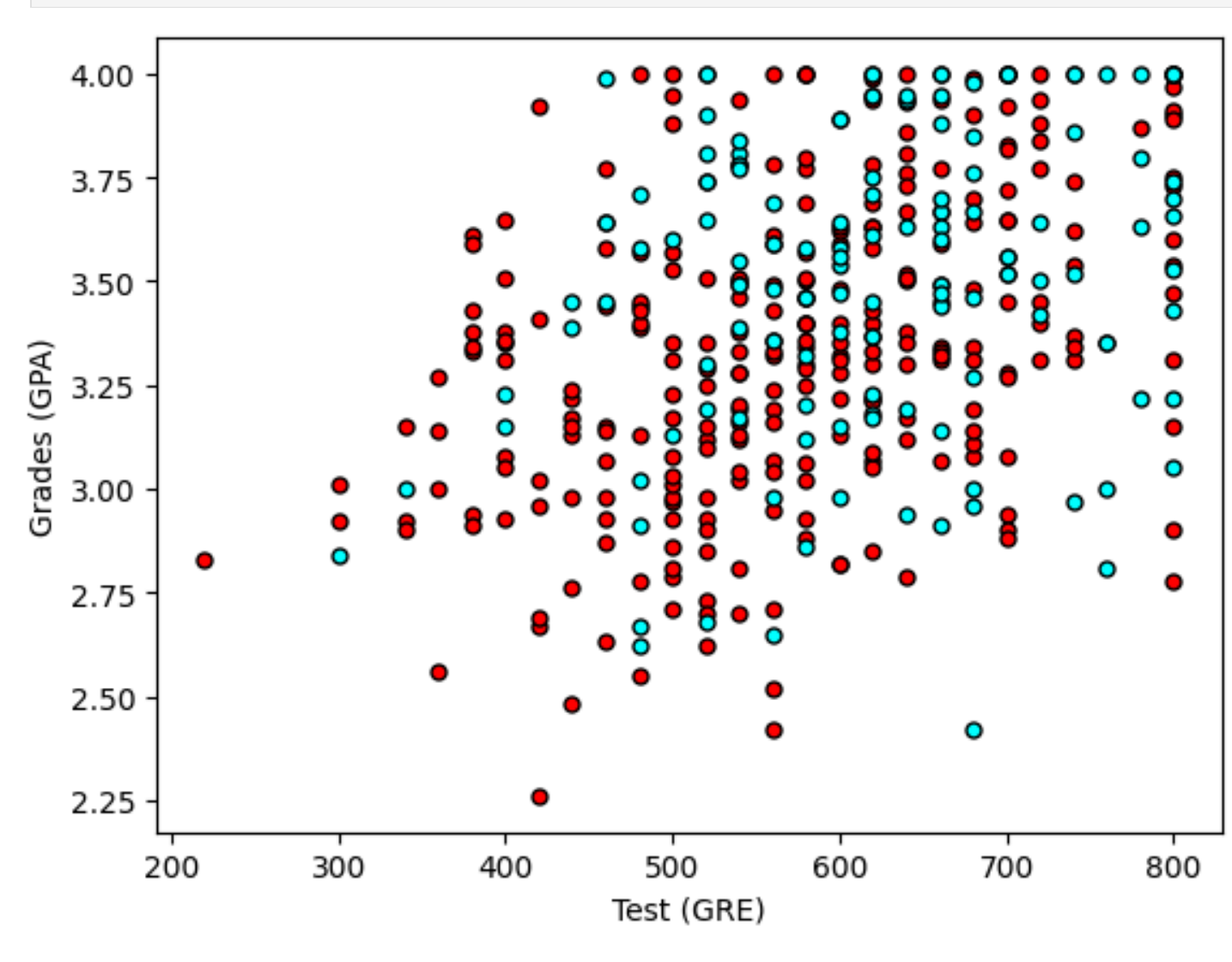
First let's make a plot of our data to see how it looks. In order to have a 2D plot, let's ingore the rank.

In [86]:

```
# %matplotlib inline
import matplotlib.pyplot as plt

# Function to help us plot
def plot_points(data):
    X = np.array(data[["gre","gpa"]])
    y = np.array(data["admit"])
    admitted = X[np.argwhere(y==1)]
    rejected = X[np.argwhere(y==0)]
    plt.scatter([s[0][0] for s in rejected], [s[0][1] for s in rejected], s = 25, color = 'red', edgecolor = 'k')
    plt.scatter([s[0][0] for s in admitted], [s[0][1] for s in admitted], s = 25, color = 'cyan', edgecolor = 'k')
    plt.xlabel('Test (GRE)')
    plt.ylabel('Grades (GPA)')

# Plotting the points
plot_points(data)
plt.show()
```

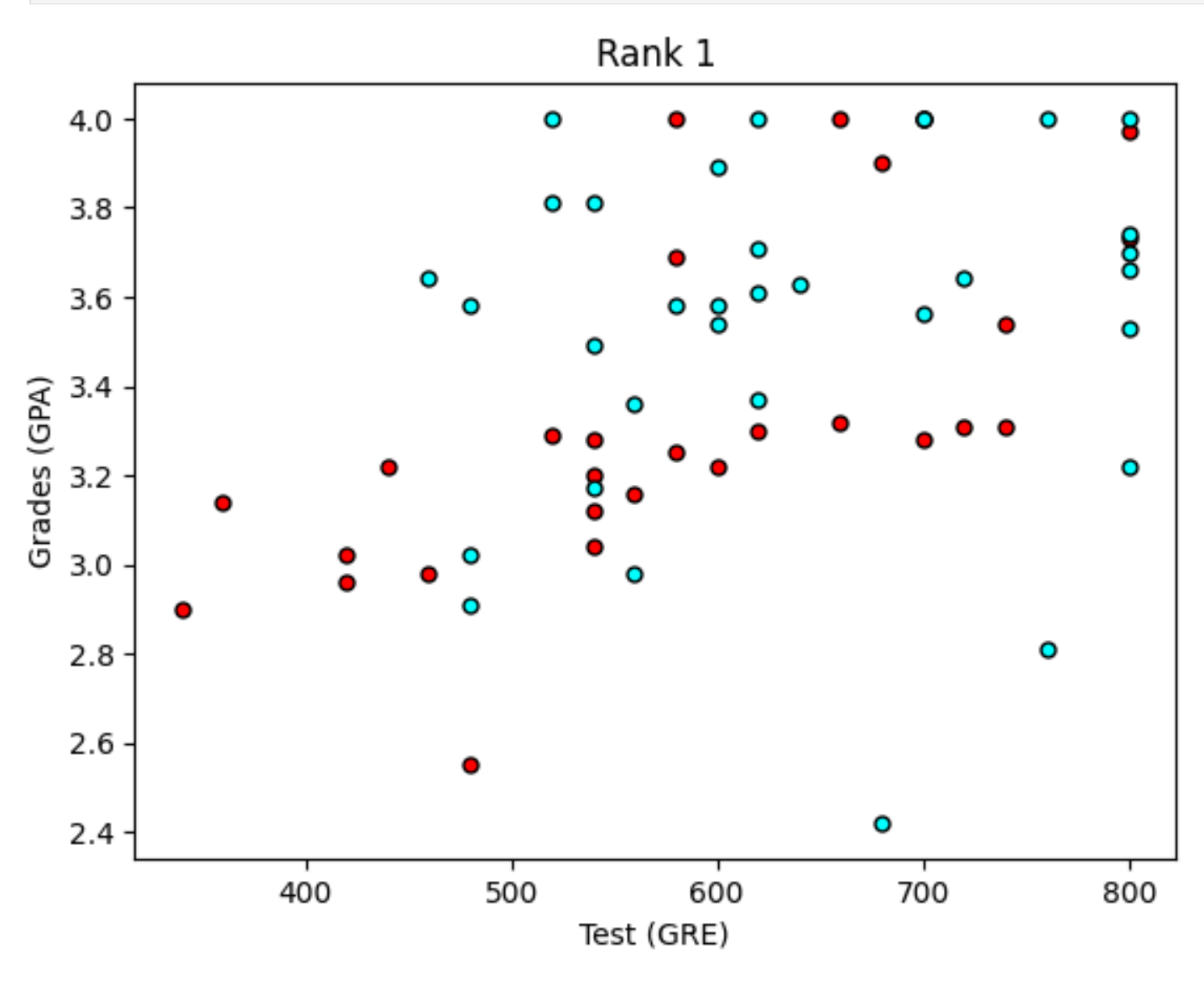


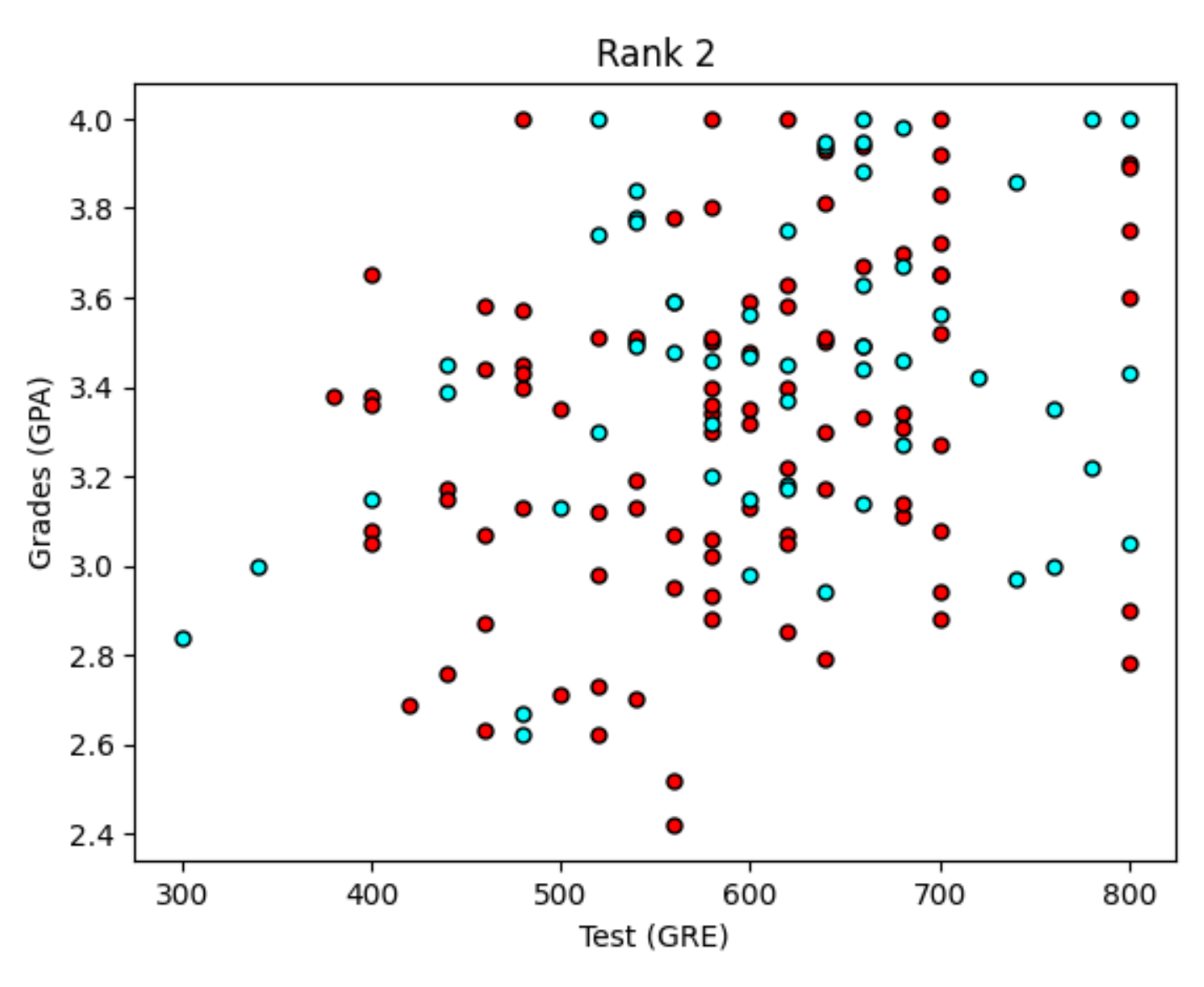
Roughly, it looks like the students with high scores in the grades and test passed, while the ones with low scores didn't, but the data is not as nicely separable as we hoped it would. Maybe it would help to take the rank into account? Let's make 4 plots, each one for each rank.

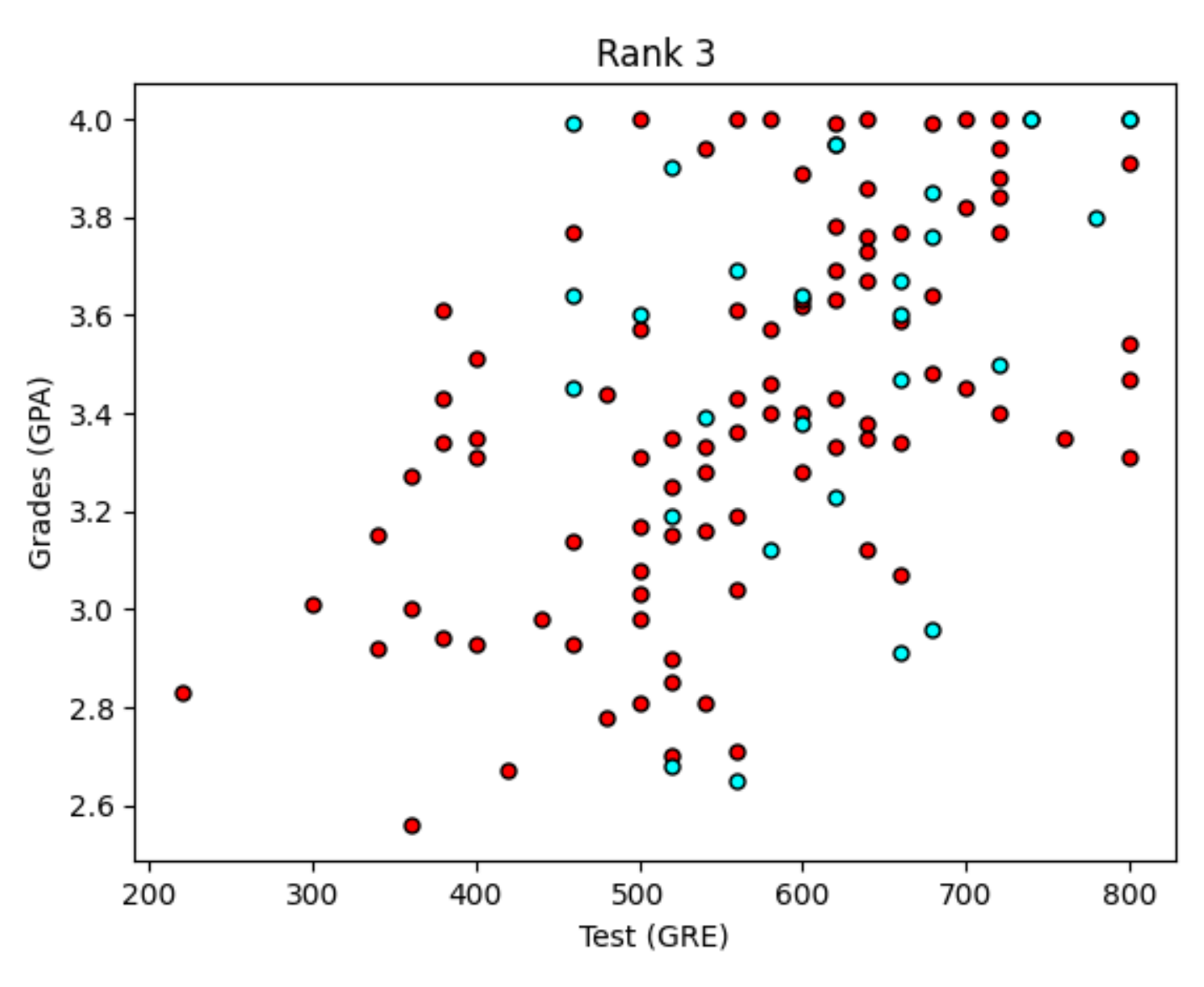
In [87]:

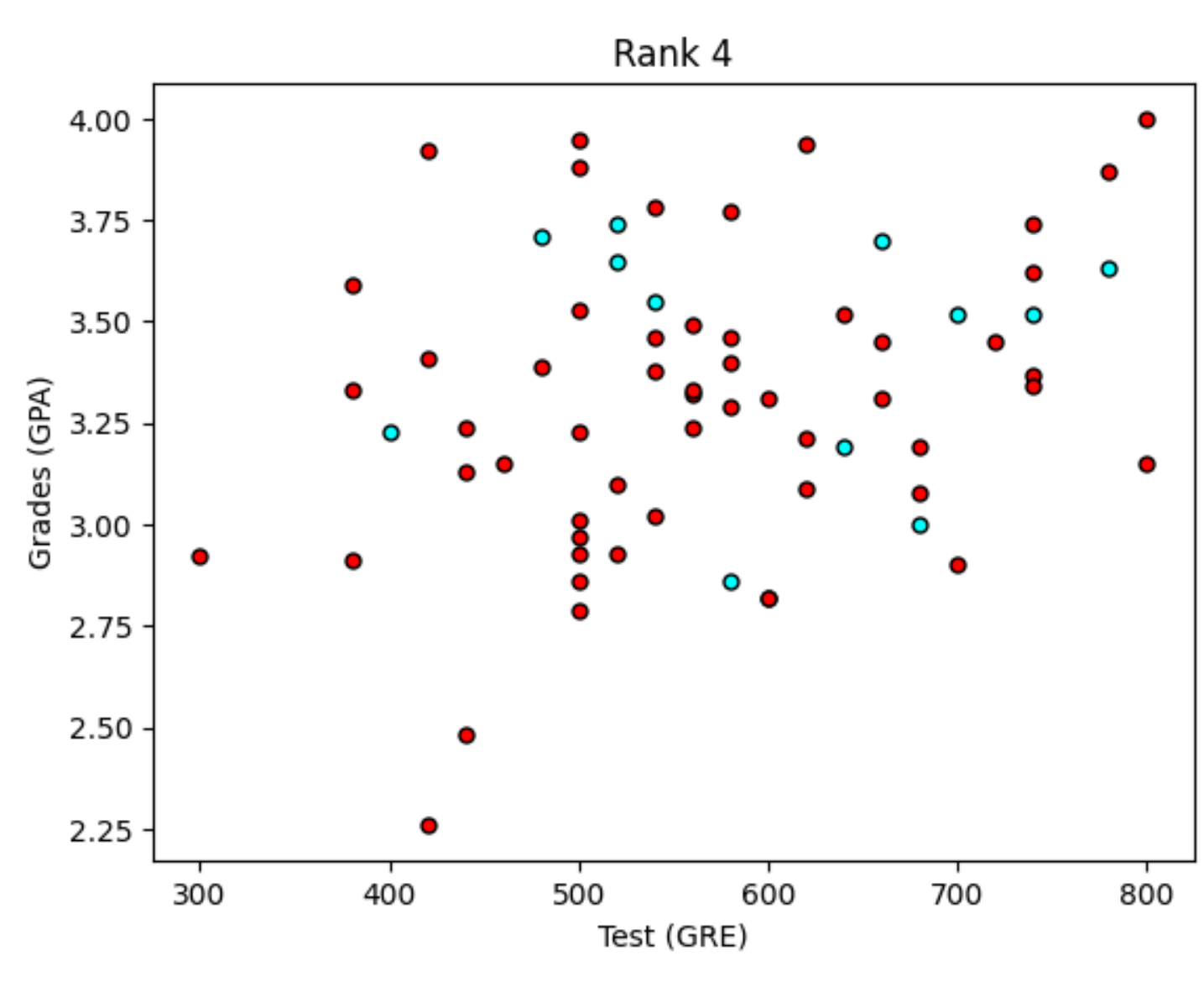
```
# Separating the ranks
data_rank1 = data[data["rank"]==1]
data_rank2 = data[data["rank"]==2]
data_rank3 = data[data["rank"]==3]
data_rank4 = data[data["rank"]==4]

# Plotting the graphs
plot_points(data_rank1)
plt.title("Rank 1")
plt.show()
plot_points(data_rank2)
plt.title("Rank 2")
plt.show()
plot_points(data_rank3)
plt.title("Rank 3")
plt.show()
plot_points(data_rank4)
plt.title("Rank 4")
plt.show()
```


Rank 1


Rank 2


Rank 3


Rank 4

This looks more promising, as it seems that the lower the rank, the higher the acceptance rate. Let's use the rank as one of our inputs. In order to do this, we should one-hot encode it.

TODO: One-hot encoding the rank

Use the `get_dummies` function in pandas in order to one-hot encode the data.

Hint: To drop a column, it's suggested that you use `one_hot_data`.drop( ).

In [88]:

```
# TODO:  Make dummy variables for rank and concat existing columns
one_hot_data = pd.concat([data, pd.get_dummies(data['rank'], prefix='rank')], axis=1)

# TODO: Drop the previous rank column
one_hot_data = one_hot_data.drop('rank', axis=1)

# Print the first 10 rows of our data
one_hot_data[:10]
```

Out[88]:

	admit	gre	gpa	rank_1	rank_2	rank_3	rank_4
0	0	380	3.61	False	False	True	False
1	1	660	3.67	False	False	True	False
2	1	800	4.00	True	False	False	False
3	1	640	3.19	False	False	False	True
4	0	520	2.93	False	False	False	True
5	1	760	3.00	False	True	False	False
6	1	560	2.98	True	False	False	False
7	0	400	3.08	False	True	False	False
8	1	540	3.39	False	False	True	False
9	0	700	3.92	False	True	False	False

TODO: Scaling the data

The next step is to scale the data. We notice that the range for grades is 1.0-4.0, whereas the range for test scores is roughly 200-800, which is much larger. This means our data is skewed, and that makes it hard for a neural network to handle. Let's fit our two features into a range of 0-1, by dividing the grades by 4.0, and the test score by 800.

In [89]:

```
# Copying our data
processed_data = one_hot_data[:]

# Scaling the columns
processed_data['gre'] = processed_data['gre']/800
processed_data['gpa'] = processed_data['gpa']/4.0
processed_data[:10]
```

Out[89]:

	admit	gre	gpa	rank_1	rank_2	rank_3	rank_4
0	0	0.475	0.9025	False	False	True	False
1	1	0.825	0.9175	False	False	True	False
2	1	1.000	1.0000	True	False	False	False
3	1	0.800	0.7975	False	False	False	True
4	0	0.650	0.7325	False	False	False	True
5	1	0.950	0.7500	False	True	False	False
6	1	0.700	0.7450	True	False	False	False
7	0	0.500	0.7700	False	True	False	False
8	1	0.675	0.8475	False	False	True	False
9	0	0.875	0.9800	False	True	False	False

Splitting the data into Training and Testing

In order to test our algorithm, we'll split the data into a Training and a Testing set. The size of the testing set will be 10% of the total data.

In [90]:

```
sample = np.random.choice(processed_data.index, size=int(len(processed_data)*0.9), replace=False)
train_data, test_data = processed_data.iloc[sample], processed_data.drop(sample)

print("Number of training samples is", len(train_data))
print("Number of testing samples is", len(test_data))
print(train_data[:10])
print(test_data[:10])
```

```
Number of training samples is 360
Number of testing samples is 40
     admit    gre     gpa  rank_1  rank_2  rank_3  rank_4
172      0  0.850  0.8700   False   False    True   False
137      0  0.875  1.0000   False   False    True   False
126      1  0.750  0.8850    True   False   False   False
94       1  0.825  0.8600   False    True   False   False
72       0  0.600  0.8475   False   False   False    True
33       1  1.000  1.0000   False   False    True   False
380      0  0.875  0.9125   False    True   False   False
223      0  1.000  0.8675   False   False    True   False
307      0  0.725  0.8775   False    True   False   False
227      0  0.675  0.7550   False   False   False    True
    admit    gre     gpa  rank_1  rank_2  rank_3  rank_4
20      0  0.625  0.7925   False   False    True   False
21      1  0.825  0.9075   False    True   False   False
48      0  0.550  0.6200   False   False   False    True
50      0  0.800  0.9650   False   False    True   False
54      0  0.825  0.8350   False   False    True   False
58      0  0.500  0.9125   False    True   False   False
87      0  0.750  0.8700   False    True   False   False
99      0  0.500  0.8275   False   False    True   False
130     1  0.775  0.7925   False    True   False   False
134     0  0.700  0.7375   False    True   False   False
```

Splitting the data into features and targets (labels)

Now, as a final step before the training, we'll split the data into features (X) and targets (y).

In [91]:

```
features = train_data.drop('admit', axis=1).astype(float)
targets = train_data['admit']
features_test = test_data.drop('admit', axis=1).astype(float)
targets_test = test_data['admit']

print(features[:10])
print(targets[:10])
```

```
       gre     gpa  rank_1  rank_2  rank_3  rank_4
172  0.850  0.8700     0.0     0.0     1.0     0.0
137  0.875  1.0000     0.0     0.0     1.0     0.0
126  0.750  0.8850     1.0     0.0     0.0     0.0
94   0.825  0.8600     0.0     1.0     0.0     0.0
72   0.600  0.8475     0.0     0.0     0.0     1.0
33   1.000  1.0000     0.0     0.0     1.0     0.0
380  0.875  0.9125     0.0     1.0     0.0     0.0
223  1.000  0.8675     0.0     0.0     1.0     0.0
307  0.725  0.8775     0.0     1.0     0.0     0.0
227  0.675  0.7550     0.0     0.0     0.0     1.0
172    0
137    0
126    1
94     1
72     0
33     1
380    0
223    0
307    0
227    0
Name: admit, dtype: int64
```

Training the 1-layer Neural Network

The following function trains the 1-layer neural network.
First, we'll write some helper functions.

In [92]:

```
# Activation (sigmoid) function
def sigmoid(x):
    return 1 / (1 + np.exp(-x))
def sigmoid_prime(x):
    return sigmoid(x) * (1-sigmoid(x))
def error_formula(y, output):
    return - y*np.log(output) - (1 - y) * np.log(1-output)
```

TODO: Backpropagate the error

Now it's your turn to shine. Write the error term. Remember that this is given by the equation

$$(y - \hat{y})x$$

for binary cross entropy loss function and

$$(y - \hat{y})\sigma'(x)x$$

for mean square error.

In [93]:

```
def error_term_formula(x, y, output):
#    for binary cross entropy loss
    return (y-output)*x
#    for mean square error
#    return (y - output)*sigmoid_prime(x)*x
```

In [94]:

```
# Neural Network hyperparameters
epochs = 1000
learnrate = 0.0001

# Training function
def train_nn(features, targets, epochs, learnrate):

    # Use to same seed to make debugging easier
    np.random.seed(42)

    n_records, n_features = features.shape
    last_loss = None

    # Initialize weights
    weights = np.random.normal(scale=1 / n_features**.5, size=n_features)

    for e in range(epochs):
        del_w = np.zeros(weights.shape)
        for x, y in zip(features.values, targets):
            # Loop through all records, x is the input, y is the target

            # Activation of the output unit
            #   Notice we multiply the inputs and the weights here
            #   rather than storing h as a separate variable
            output = sigmoid(np.dot(x, weights))

            # The error term
            error_term = error_term_formula(x, y, output)

            # The gradient descent step, the error times the gradient times the inputs
            del_w += error_term * x

        # Update the weights here. The learning rate times the
        # change in weights
        # don't have to divide by n_records since it is compensated by the learning rate
        weights += learnrate * del_w #/ n_records

        # Printing out the mean square error on the training set
        if e % (epochs / 10) == 0:
            out = sigmoid(np.dot(features, weights))
            loss = np.mean(error_formula(targets, out))
            print("Epoch:", e)
            if last_loss and last_loss < loss:
                print("Train loss: ", loss, "  WARNING - Loss Increasing")
            else:
                print("Train loss: ", loss)
            last_loss = loss
            print("=========")
    print("Finished training!")
    return weights

weights = train_nn(features, targets, epochs, learnrate)
```

```
Epoch: 0
Train loss:  0.7465918770816229
=========
Epoch: 100
Train loss:  0.6341866823616823
=========
Epoch: 200
Train loss:  0.6181891179202362
=========
Epoch: 300
Train loss:  0.6132412860190256
=========
Epoch: 400
Train loss:  0.6103842520384822
=========
Epoch: 500
Train loss:  0.6083272042308147
=========
Epoch: 600
Train loss:  0.6067461535140861
=========
Epoch: 700
Train loss:  0.6054960887694494
=========
Epoch: 800
Train loss:  0.6044884488413848
=========
Epoch: 900
Train loss:  0.6036622694203103
=========
Finished training!
```

Calculating the Accuracy on the Test Data

In [95]:

```
# Calculate accuracy on test data
test_out = sigmoid(np.dot(features_test, weights))
predictions = test_out > 0.5
accuracy = np.mean(predictions == targets_test)
print("Prediction accuracy: {:.3f}".format(accuracy))
```

```
Prediction accuracy: 0.700
```